



INDIA.RUNS

Build what next India runs on

Team Name : NEURAL RUNNERS

Team Leader Name : M . Venkata Kartheek

Problem Statement :

Develop an AI-powered candidate ranking system that goes beyond keyword matching by understanding job requirements and candidate profiles contextually. The system should intelligently rank the best-fit candidates using profile, career, and behavioral signals with high accuracy and low latency.

Solution Overview

- Our Intelligent Candidate Discovery System uses a 6-stage multi-agent AI pipeline to match candidates with job descriptions. Built with React/Vite, Python/Flask, Supabase (pgvector), and Groq Llama 3.3 70B, it semantically analyzes, evaluates, and ranks candidates through specialized AI agents, producing a highly accurate shortlist.
- Traditional systems rely on basic keyword-based similarity matching, often missing context. Our system improves accuracy by:
 - 1) Structured JD Analysis: Converts JDs into a Requirement Specification before matching.
 - 2) Multi-Agent Evaluation: Uses specialized AI agents to assess different candidate attributes.
 - 3) Self-Correcting Critic: Verifies and refines AI-generated scores to reduce errors.
 - 4) Reliable Fallbacks: Uses deterministic methods when LLM services are unavailable.

JD Understanding & Candidate Evaluation

- Our Stage 1 JD Decomposer Agent converts the Job Description into a structured requirement specification by extracting:
 - 1) Must-Have Skills
 - 2) Nice-to-Have Skills
 - 3) Years of Experience
 - 4) Domain Knowledge
 - 5) Dynamic Requirement Weights
- We evaluate candidate fit semantically rather than through exact keyword matches. The most important signals are:

Semantic Claim Extraction: We evaluate what the candidate has *actually done* (extrapolated from profile activity) rather than just looking for buzzwords.

Behavioral Signals: We isolate soft skills and behavioral traits into an independent behavioral_score.

Domain Fit: We evaluate whether their past projects align with the industry domain of the JD, even if the tech stack is slightly different. By using FAISS and all-MiniLM-L6-v2 dense vector embeddings

Ranking Methodology

Stage 2 - Retrieve:

- FAISS semantic vector search
- Retrieves top 20–30% most relevant candidates
- Reduces LLM computation

Stage 4- Validate

- Critic Agent reviews scores
- Applies penalties for weak or inconsistent profiles
- LLM: Groq +Llama 3.3-70B-Versatile
- Embeddings: all MiniLM-L6-v2

Stage 3 - Score

- Evaluates Must Have Skills
- Evaluates Experience & Behavioral Traits
- Evaluates Domain Fit

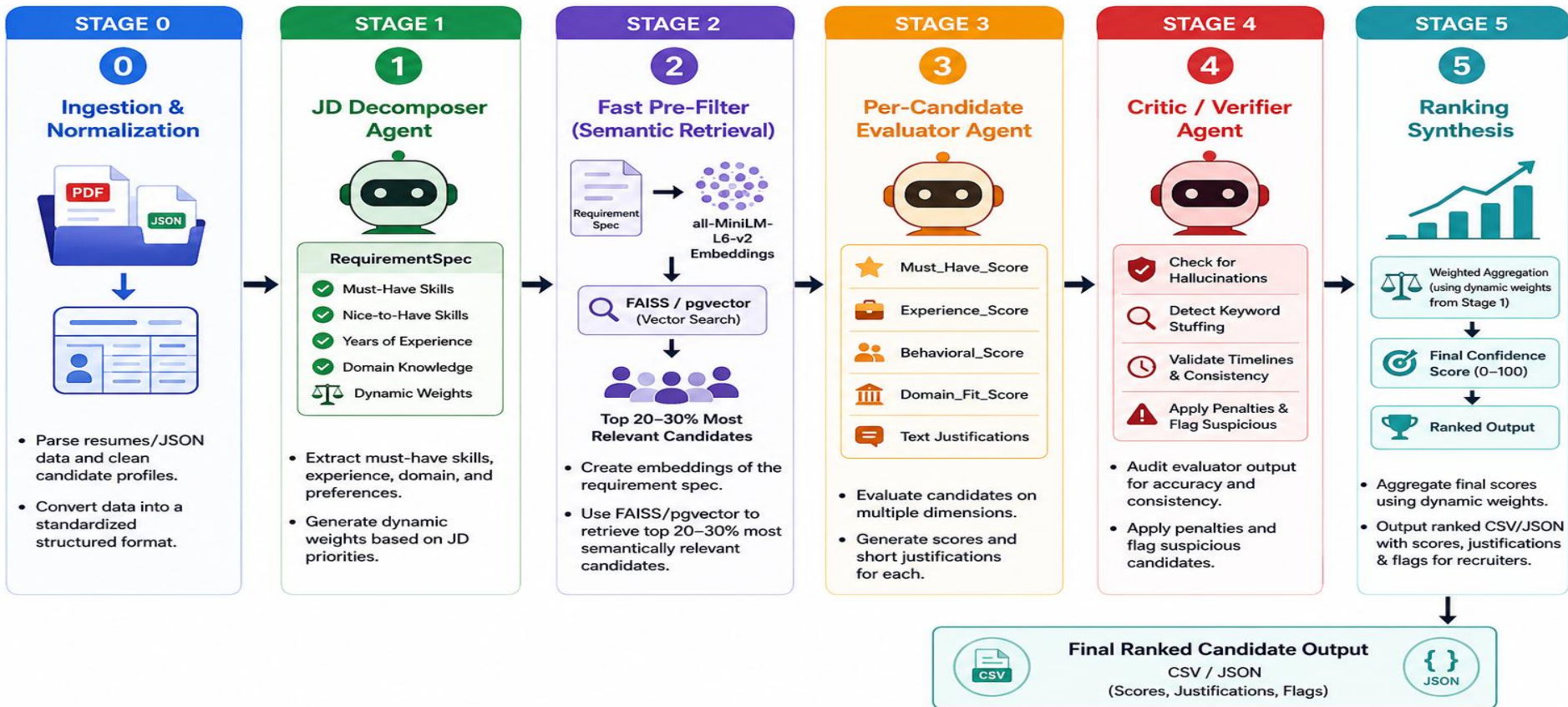
Stage 5 - Rank

- Weighted mathematical aggregation
- Dynamic weights based on the JD
- Vector Search: FAISS + pgvector
- Fallback : Rule based parsing + TF - IDF

Explainability & Data Validation

- **Transparent AI:** Each score includes a clear textual justification generated by the Evaluator Agent.
- **Explainable Rankings:** Justifications are displayed in the UI, showing why one candidate ranks higher than another.
- **Critic Agent Verification:** Independently validates scores and justifications against the candidate's raw profile.
- **Automatic Corrections:** Detects unsupported claims, applies score penalties, and rewrites inaccurate explanations.
- **Data Validation:** Normalizes profiles, flags missing or incomplete information, and uses fallback heuristics when needed.
- **Fraud Detection:** Identifies keyword stuffing and unrealistic experience, applies penalties, and marks profiles with a Suspicious flag.

Complete Workflow: JD to Ranked Candidate Output



System Architecture

Our application is built on a modern, highly decoupled architecture designed for scalability and ultra-low latency AI inference:

Frontend Layer (React + Vite): A responsive, conversational UI that allows recruiters to seamlessly paste job descriptions, upload resumes, and securely manage API keys using browser local storage.

Backend API Layer (Python + Flask): A lightweight REST API running via Gunicorn. It handles document parsing (via pypdf), manages data flow, and orchestrates the complex multi-agent pipeline.

Intelligence Layer (Groq API + Llama 3.3): Instead of relying on slow, expensive models, we integrate Groq's API running llama-3.3-70b-versatile to achieve near-instantaneous agentic reasoning (Decomposer, Evaluator, and Critic agents).

Vector & Data Storage (Supabase + pgvector): Candidate metadata is securely stored in a Supabase PostgreSQL database. We use the pgvector extension combined with FAISS and II-MiniLM-L6-v2 dense embeddings to perform lightning-fast semantic similarity searches across thousands of candidate profiles before the LLM step even begins.

Results & Performance

- Eliminates false positives by detecting keyword stuffing and inconsistent profiles.
- Provides explainable rankings with sub-scores and AI-generated justifications.
- Uses semantic matching to identify qualified candidates beyond exact keyword matches.

Runtime & Compute Optimization

- FAISS pre-filter evaluates only the top 20% of relevant candidates, reducing LLM workload.
- Precomputed embeddings enable near-instant candidate retrieval with minimal computation.
- Groq-powered low-latency inference with deterministic TF-IDF fallback ensures fast, reliable execution within runtime limits.

Github Video :

Uploaded in git :

<https://github.com/Kartheek-3/CandidateRankingSystem>

Technologies Used

- Groq API (Llama 3.3 70B) – Ultra-fast LLM inference for low-latency multi-agent reasoning.
- FAISS + all-MiniLM-L6-v2 – Fast semantic vector search for accurate candidate retrieval.
- Python + Flask – Lightweight backend with strong AI/ML ecosystem support.
- React + Vite – Modern frontend for responsive UI and fast development.
- Supabase + PostgreSQL (pgvector) – Scalable database with native vector search support.
- Pandas, NumPy & Sentence-Transformers – Efficient data processing, embedding generation, and semantic analysis.



INDIA.RUNS

Build what next India runs on

THANK YOU